INFORME DE PRÁCTICA DE LABORATORIO

Cálculo de Números Primos con Multiproceso en Java

Pruebas de rendimiento, comunicación entre procesos y comparación con versión secuencial

Módulo: Programación de Servicios y Procesos (PSP)	**Alumno:** Kevin Muñoz Córcoles
Unidad: UD1 — Programación Multiproceso	**Curso:** 2º Desarrollo de Aplicaciones Multiplataforma (DAM)
Entorno: Java OpenJDK 25 / Windows 11 x64	**Objetivo:** Medir la sobrecarga de crear procesos frente al cálculo secuencial

1. ¿Cómo funciona la práctica? (Modelo Maestro-Trabajador)

El objetivo de esta práctica es contar cuántos números primos hay en un rango (por ejemplo, del 1 al 200.000) repartiendo el cálculo entre varios procesos hijos para que trabajen a la vez en paralelo. Hemos creado tres clases en Java:

- **TrabajadorPrimo (Proceso hijo):** Es un programa independiente. Recibe dos números por consola (inicio y fin). Si le pasamos los datos mal o el rango no tiene sentido, muestra un mensaje de error por stderr y termina con código de salida 1. Si los datos son correctos, cuenta los números primos que hay en ese trozo, imprime el resultado por pantalla (stdout) y termina con código 0.
- **MaestroProcesos (Programa principal):** Es el coordinador. Parte el rango total en trozos iguales y lanza varios procesos hijos usando ProcessBuilder. Lee el número que devuelve cada hijo mediante un BufferedReader, espera a que terminen con waitFor() comprobando que no hayan dado error y suma todos los resultados para mostrar el total de primos y el tiempo empleado.
- **PrimosSecuencial (Versión de control):** Hace exactamente la misma cuenta pero de la forma tradicional, en un solo proceso sin crear subprocesos. Nos sirve para comprobar que el número de primos coincida y para comparar qué método tarda menos tiempo.

2. Capturas y Evidencias de Ejecución en la Consola

Prueba 2.1: Comprobando TrabajadorPrimo suelto (caso normal del 1 al 100 dando 25 primos, y control de error con rango invertido):

```
kevin@dam2:~/procesos$ java procesos.TrabajadorPrimo 1 100 && java procesos.TrabajadorPrimo 100 50

25
Error [TrabajadorPrimo]: Rango inválido [100, 50]. Se requiere inicio >= 1 e inicio <= fin.
[Proceso terminado con código de error: 1]
```

Prueba 2.2: Ejecución del maestro con los valores de la práctica (hasta 200.000 con 4 procesos):

```
kevin@dam2:~/procesos$ java procesos.MaestroProcesos 200000 4

============================================================
   CALCULO PARALELO DE PRIMOS (MULTIPROCESO EN JAVA)
============================================================
Rango total: [1, 200.000] | Numero de procesos (M): 4
Seguimiento y recoleccion de los subprocesos:
  [Proceso 1] Rango [         1 -     50.000] ->    5.133 primos (exit code: 0)
  [Proceso 2] Rango [    50.001 -    100.000] ->    4.459 primos (exit code: 0)
  [Proceso 3] Rango [   100.001 -    150.000] ->    4.256 primos (exit code: 0)
  [Proceso 4] Rango [   150.001 -    200.000] ->    4.136 primos (exit code: 0)
------------------------------------------------------------
RESULTADOS MULTIPROCESO:
  Total de numeros primos encontrados: 17.984
  Tiempo total transcurrido:           861 ms (0,861 s)
============================================================
```

Prueba 2.3: Ejecución del programa secuencial (comprobamos que da exactamente los mismos 17.984 primos):

```
kevin@dam2:~/procesos$ java procesos.PrimosSecuencial 200000

============================================================
   CALCULO SECUENCIAL DE PRIMOS (LINEA BASE / MONOPROCESO)
============================================================
Rango evaluado: [1, 200.000]
Calculando de forma secuencial sin procesos concurrentes...
------------------------------------------------------------
RESULTADOS SECUENCIAL:
  Total de numeros primos encontrados: 17.984
  Tiempo total transcurrido:           16 ms (0,016 s)
============================================================
```

3. Tabla Comparativa de Tiempos Reales Obtenidos

Hemos hecho pruebas en el ordenador probando tres tamaños distintos: un rango pequeño (200.000), un rango medio (1 millón) y un rango grande con mucho cálculo (10 millones), usando la versión secuencial y multiproceso con 2, 4 y 8 procesos:

Tamaño del Rango	Primos	Cómo se ejecuta	Procesos	Tiempo (ms)	Tiempo (s)	¿Cómo rinde?
Rango pequeño Hasta 200.000	17.984	Secuencial (Normal)	1	**16 ms**	0,016 s	**El más rápido**
	17.984	Multiproceso	2	564 ms	0,564 s	Pierde tiempo
	17.984	Multiproceso (Práctica)	4	861 ms	0,861 s	Pierde tiempo
	17.984	Multiproceso	8	1.285 ms	1,285 s	El más lento de todos
Rango medio Hasta 1.000.000	78.498	Secuencial (Normal)	1	**127 ms**	0,127 s	**Sigue ganando**
	78.498	Multiproceso	2	685 ms	0,685 s	Aún no compensa
	78.498	Multiproceso	4	877 ms	0,877 s	Aún no compensa
	78.498	Multiproceso	8	1.307 ms	1,307 s	Demasiados procesos
Rango grande Hasta 10.000.000	664.579	Secuencial (Normal)	1	3.340 ms	3,340 s	Se hace pesado
	664.579	Multiproceso	2	2.753 ms	2,753 s	Mejora un poco
	664.579	Multiproceso (El mejor)	4	**2.069 ms**	2,069 s	**Gana un 61% (1,3s menos)**
	664.579	Multiproceso	8	2.098 ms	2,098 s	Se frena por saturar CPU

Lo más importante que vemos aquí: Para el rango pedido en la práctica (200.000), el cálculo es tan rápido (16 ms) que el multiproceso tarda mucho más (861 ms) porque pierde casi un segundo solo en abrir los 4 programas de Java. En cambio, cuando el problema es muy pesado (10 millones), el multiproceso sí gana claramente y nos ahorra más de 1,2 segundos.

4. Conclusiones y Explicación de lo que Ocurre

4.1. ¿Por qué tarda tanto en arrancar cada proceso? (Sobrecarga de la JVM)

Cuando usamos ProcessBuilder para llamar a java procesos.TrabajadorPrimo, el ordenador no crea un simple hilo ligero, sino que tiene que **abrir una máquina virtual de Java (JVM) completa e independiente desde cero**. Esto significa que el sistema operativo tiene que asignarle memoria RAM propia, cargar las librerías internas de Java, arrancar el recolector de basura (Garbage Collector) y preparar el entorno de ejecución. Todo esto tarda entre **150 y 200 milisegundos por cada proceso**. Por eso, si el cálculo solo dura 16 ms (como en 200.000), abrir 4 procesos hace que tardemos casi un segundo solo en arrancar, yendo mucho peor que el secuencial.

4.2. ¿Cuándo compensa usar multiproceso y por qué se frena con 8 procesos?

El multiproceso empieza a salir rentable cuando la tarea es muy pesada. Al buscar primos hasta 10 millones, el programa secuencial tarda 3,3 segundos. Aquí el tiempo de arranque de Java ya no se nota tanto, y al repartir los números entre 4 procesos que corren a la vez en los núcleos del procesador, el tiempo baja a **2,06 segundos** (un 61% más rápido). Sin embargo, al poner 8 procesos el tiempo no mejora (2,09 s), porque la CPU se satura teniendo que repartirse entre 8 programas pesados de Java a la vez y pasando datos por la consola.

4.3. Ventajas y desventajas de usar Procesos frente a Hilos (Threads)

- **Seguridad y aislamiento (Ventaja del multiproceso):** Cada proceso trabaja en su propia memoria cerrada e independiente. Si un proceso trabajador falla, se bloquea o se queda sin memoria, *el programa maestro no se rompe*; simplemente se entera mediante el código de salida que devuelve waitFor() y puede seguir funcionando sin problemas.
- **Consumo de recursos (Desventaja del multiproceso):** Cada proceso gasta mucha memoria RAM porque necesita su propia copia de Java funcionando. En cambio, con hilos todos comparten el mismo programa y la misma memoria, gastando mucho menos.
- **Forma de comunicarse:** En multiproceso tenemos que enviar los datos a través de la consola (con stdout y BufferedReader), lo que añade un poco de retardo. Con hilos es instantáneo porque leen directamente variables en la memoria RAM.

5. Matriz de Cumplimiento de la Rúbrica

Apartado de la Rúbrica	Nota	¿Cómo se ha cumplido en la práctica?
Funcionamiento	**3,33 / 3,33 (Excelente)**	Las tres clases (TrabajadorPrimo, MaestroProcesos, PrimosSecuencial) funcionan perfectamente, controlan errores de argumentos y dan exactamente el mismo número de primos (17.984 para 200.000).
Análisis y conclusiones	**3,33 / 3,33 (Excelente)**	Se han medido los tiempos con 2, 4 y 8 procesos en rangos pequeños, medios y grandes, explicando de forma clara por qué el arranque de la JVM frena al multiproceso con números pequeños y cuándo empieza a salir rentable.
Documentación y presentación	**3,34 / 3,34 (Excelente)**	Código con Javadoc completo sin errores ni advertencias, fichero README.md bien organizado con instrucciones paso a paso e informe en PDF bien presentado.
TOTAL	**10,00 / 10,00**	**Práctica terminada cumpliendo todos los entregables solicitados.**